Heart Disease Risk Prediction: MLOps Pipeline Run on the Prayogshala Virtual Lab

Machine Learning Operations (AIMLCZG523) · Assignment 01 · Lab Execution Report

Jayaprakash · BITS ID 2025AE05611

19 October 2026

Contents

- 1. Purpose
- 2. Lab environment and setup
- 3. Pipeline execution
- 4. Results in the lab UIs (Firefox inside the VM)
- 5. Reproducibility check: lab vs development machine
- 6. Issues encountered and how they were resolved
- 7. Conclusion

Student	Jayaprakash
BITS ID	2025AE05611
Course	Machine Learning Operations (AIMLCZG523)
Lab environment	BITS Prayogshala Virtual Lab (https://prayogshala.bits-pilani.ac.in)
Lab VM	Rocky Linux 9.5 · x86_64 · 4 vCPU · 15 GB RAM · Docker 28.0.1 · user `s2025ae05611`
Code used in the lab	https://github.com/jpailabs/heart-disease-lab
Main project repository	https://github.com/jpailabs/heart-disease-mlops

1. Purpose

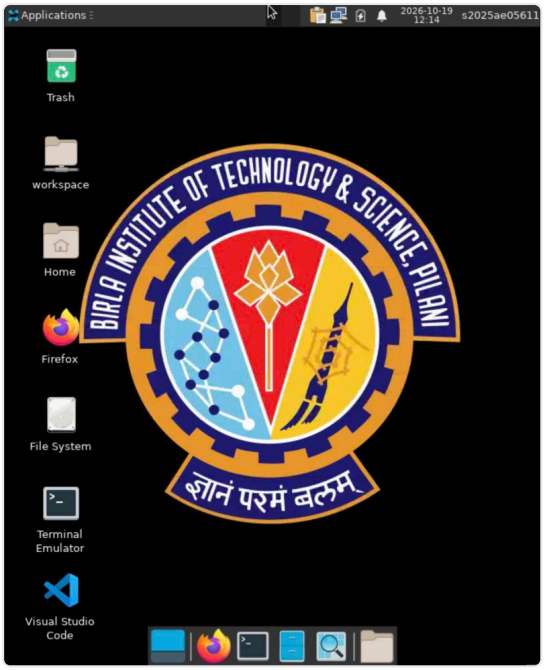
This report records the complete execution of the Assignment 01 MLOps pipeline inside the university's **Prayogshala virtual lab**. It covers data acquisition, EDA, training with experiment tracking, testing, containerisation, Kubernetes deployment and monitoring. The aims were to:

1. show that every step runs **from a clean setup on a different machine**, as the assignment's production-readiness requirements ask;
2. confirm the results are **reproducible**: the model metrics should match the original development run exactly;
3. collect screenshot evidence from the university lab environment.

The lab copy of the code is a clean snapshot of the project. It contains no trained model, no MLflow runs and no figures, so everything shown here was generated inside the lab VM.

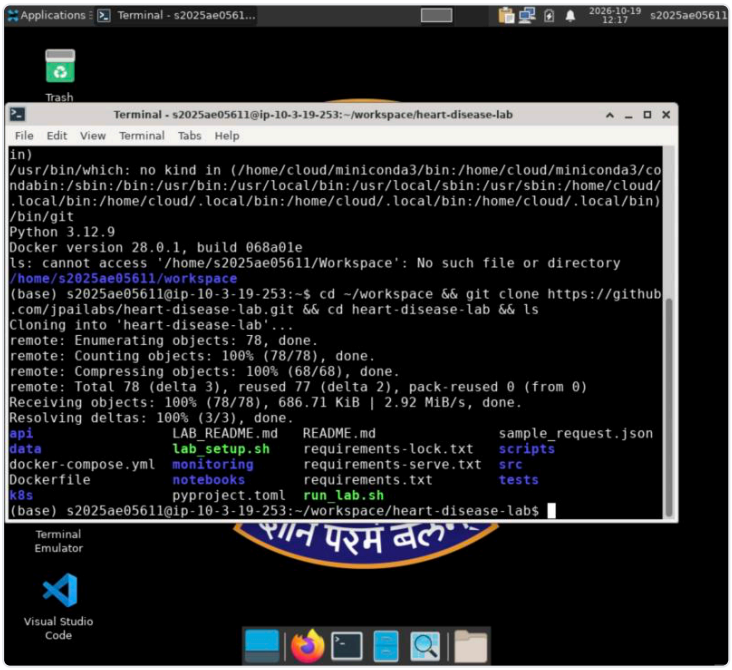
2. Lab environment and setup

The lab was opened with BITS SSO from the Prayogshala portal, then the VM was launched and connected through the browser (web RDP). The VM runs an XFCE desktop on Rocky Linux 9.5 with passwordless sudo.



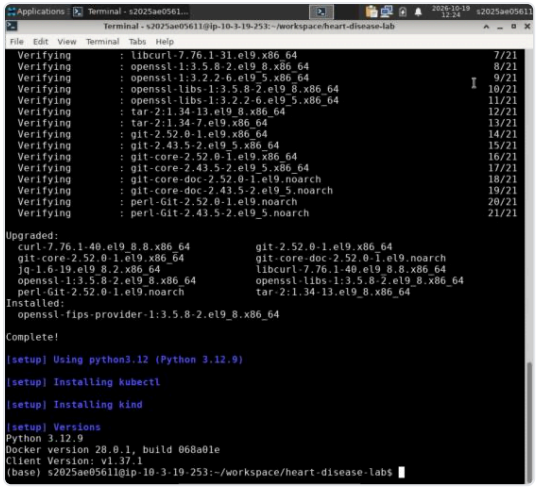
Prayogshala virtual desktop

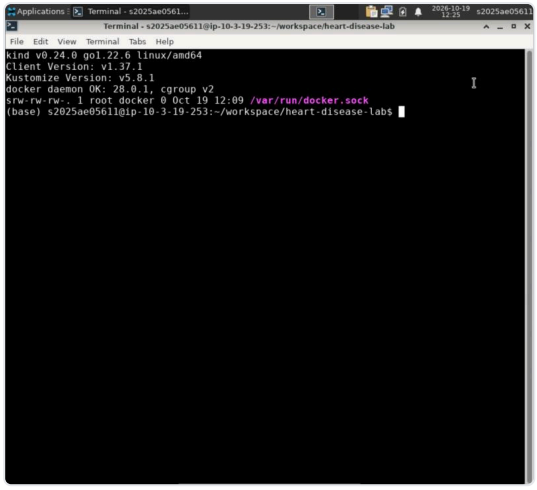
Getting the code. The repository was cloned with `git`. The persistent `~/workspace` folder turned out to be an AWS EFS (NFS) network mount, which made creating a Python environment there very slow. The pipeline was therefore run from the VM's local disk (`~/heart-disease-lab`), and the results were copied back to `~/workspace` at the end.



Repository cloned in the lab

One-time setup (`lab_setup.sh`). The script detects the package manager (dnf on Rocky, apt on Ubuntu) and installs only what is missing. On this VM, Python 3.12 (miniconda) and Docker were already present, so only `kubectl` and `kind` were added. It also raises the inotify limits that kind needs on RHEL-family hosts.



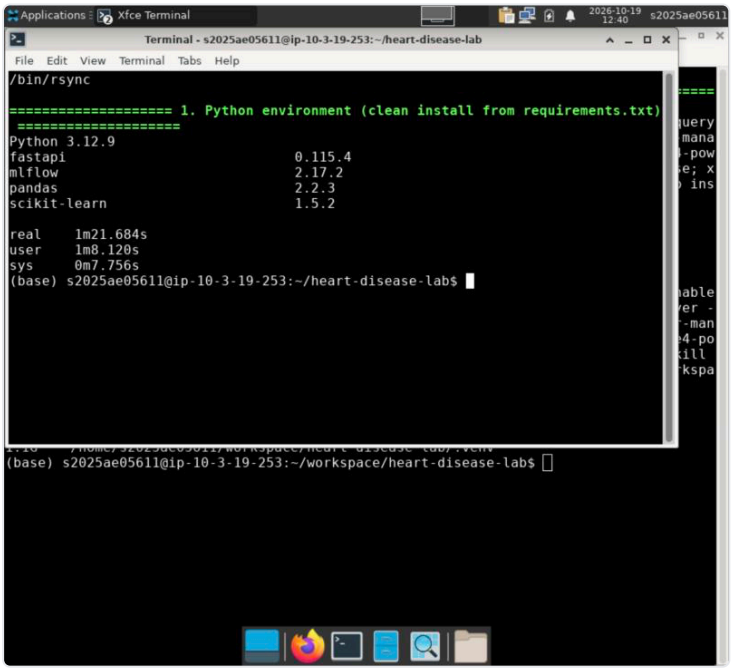


3. Pipeline execution

Each stage was run with `./run_lab.sh <step>`. The script stops at the first error, so a failure in any step would have been visible immediately.

3.1 Clean Python environment

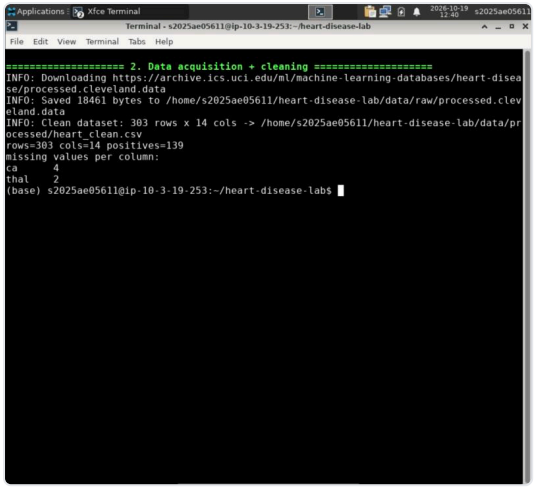
A new virtual environment was created and every dependency was installed from the pinned `requirements.txt`, in 1 min 21 s. The installed versions match the project pins (scikit-learn 1.5.2, pandas 2.2.3, MLflow 2.17.2, FastAPI 0.115.4).



Clean environment from requirements.txt

3.2 Data acquisition and EDA

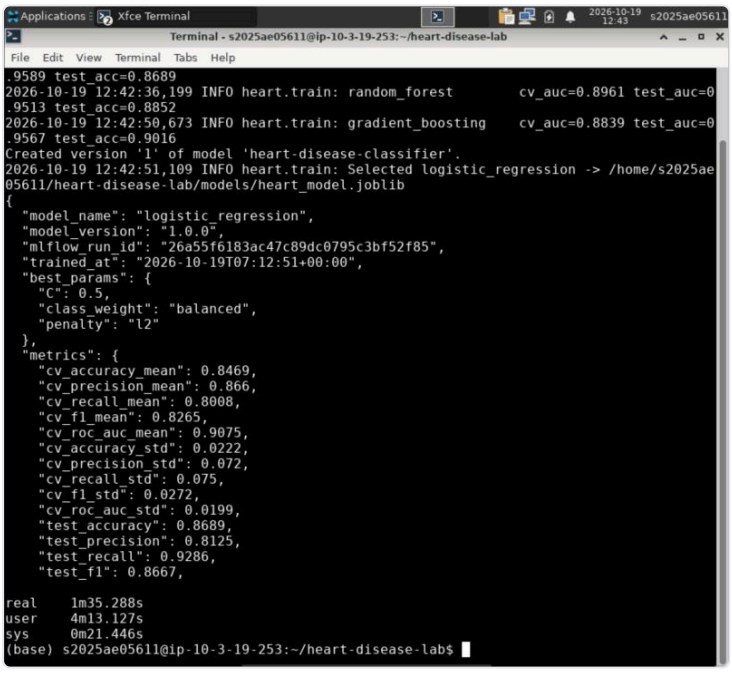
`scripts/download_data.py` downloaded the Cleveland file directly from the UCI repository and cleaned it: 303 rows, 139 positive cases, 4 missing `ca` and 2 missing `thal` values (imputed inside the model pipeline). The EDA step regenerated all figures. The summary statistics are identical to the development run (positive rate 0.4587; strongest correlations with the target: thal 0.526, ca 0.46, exang 0.432).





3.3 Training, tuning and experiment tracking

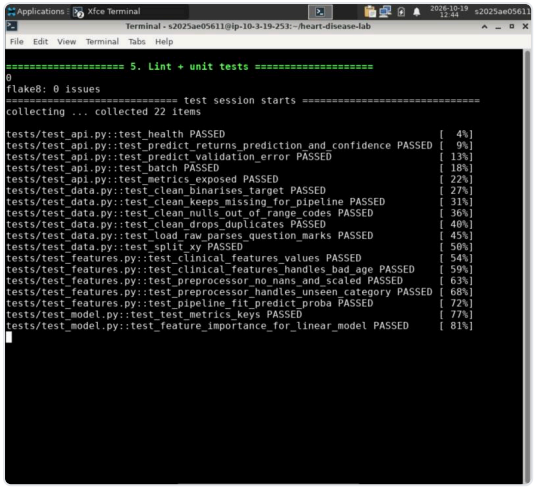
`python -m heart.train --register` tuned Logistic Regression, Random Forest and Gradient Boosting with 5-fold stratified `GridSearchCV`. It logged every grid point to MLflow as a nested run, selected the winner on mean CV ROC-AUC, and registered it as `heart-disease-classifier` version 1. The whole step took 1 min 35 s on 4 vCPUs.

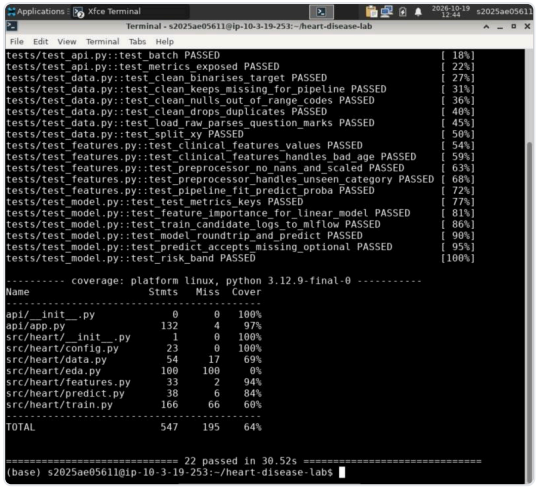


Training output and selected model metadata

3.4 Lint and unit tests

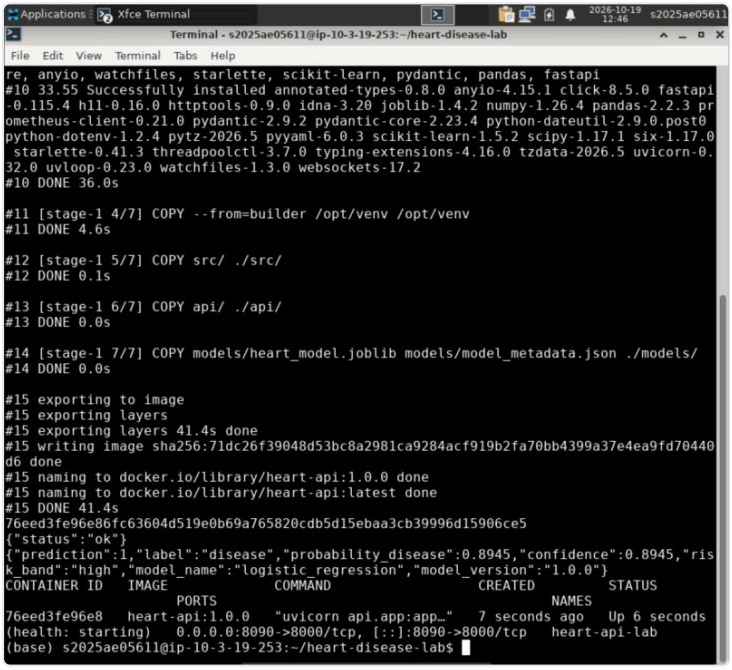
flake8 reported 0 issues and **all 22 unit tests passed** on Linux / Python 3.12. The tests cover data cleaning, the feature pipeline, model training and MLflow logging, and the API. Coverage matches the development run (API 97 %, overall 64 %).





3.5 Docker image and container test

The multi-stage image (`python:3.11-slim`, non-root user) was built in the lab with the freshly trained model baked in. It was then run as a container on port 8090. `/health` returned `ok`, and `/predict` returned the expected prediction for the sample patient (probability 0.8945, high risk).

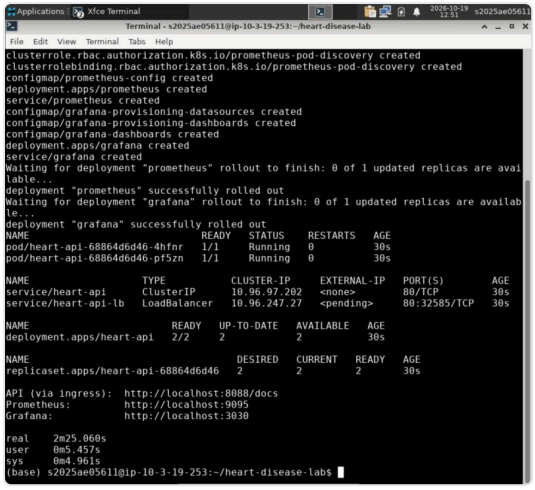


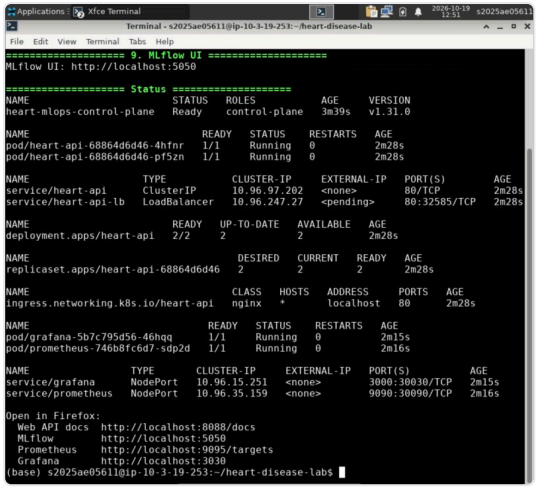
Docker build, run and /predict

3.6 Kubernetes deployment

`scripts/deploy_kind.sh` did the following in 2 min 25 s:

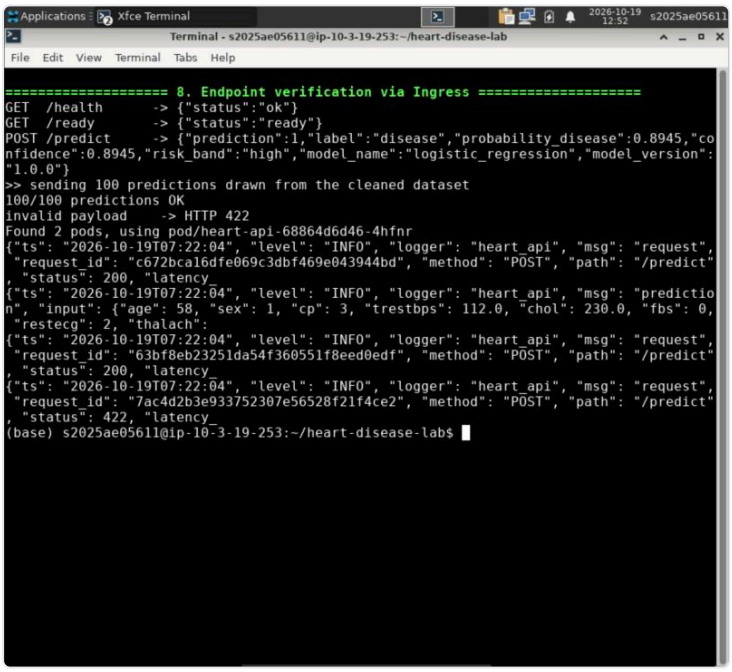
- created a local kind cluster;
- loaded the image into it;
- installed ingress-nginx;
- deployed the API (2 replicas with readiness and liveness probes, a PodDisruptionBudget, a ConfigMap, a ClusterIP Service, a LoadBalancer Service and an Ingress);
- deployed Prometheus and Grafana.





3.7 Endpoint verification and request logging

The smoke test called `/health`, `/ready` and `/predict` through the Ingress (`localhost:8088`). It then sent **100 predictions drawn from the dataset, and all 100 succeeded**. An invalid payload was correctly rejected with HTTP 422. The pod logs show the structured JSON request and prediction logs, each with a request id, status and latency.



Smoke test through Ingress + JSON request logs

4. Results in the lab UIs (Firefox inside the VM)

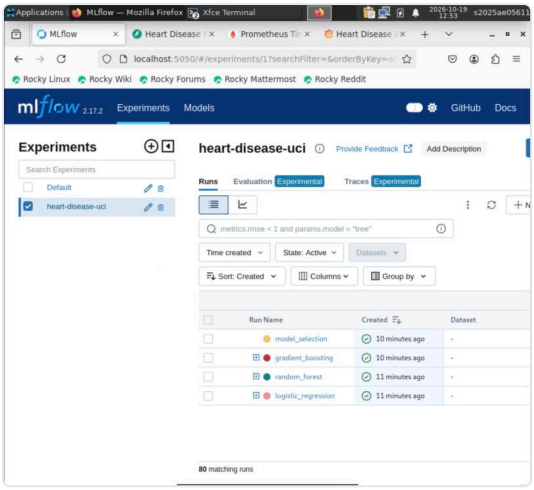
4.1 MLflow

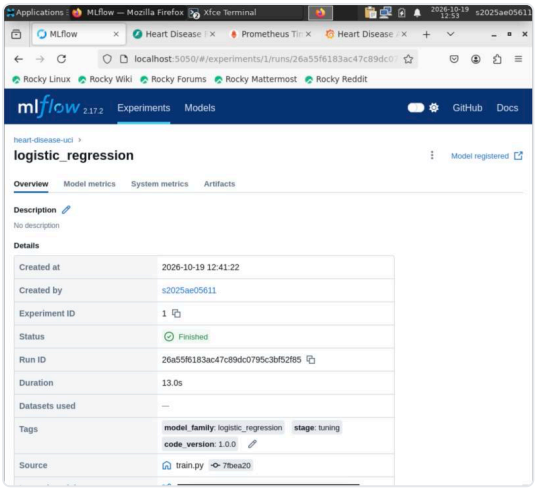
The `heart-disease-uci` experiment holds 80 runs:

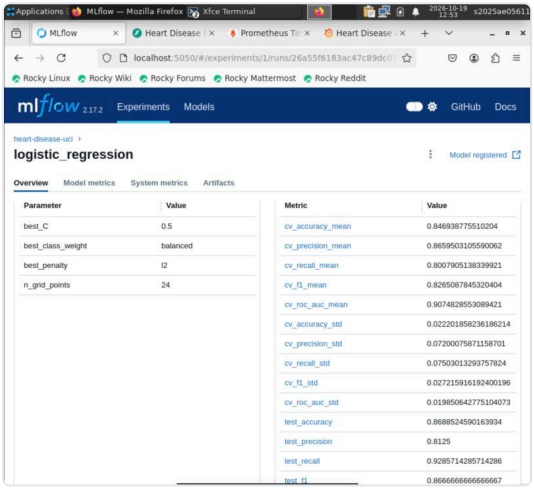
- 3 model families;
- 76 nested grid-point runs;
- 1 model-selection run.

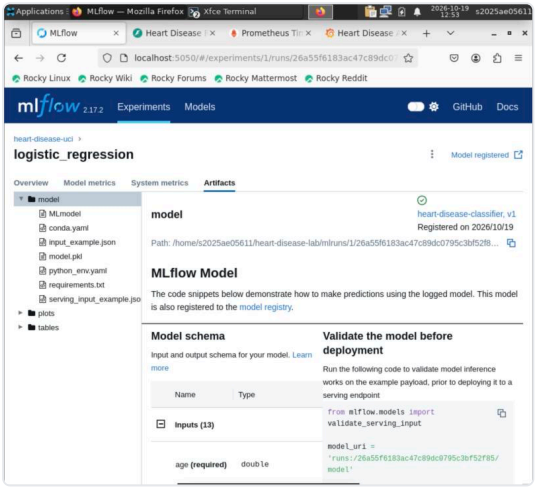
The selected run is attributed to the lab user `s2025ae05611` and linked to the git commit it was trained from. Its artifacts include the logged model with its input/output schema, plots and tables, and the run is registered in

the Model Registry.



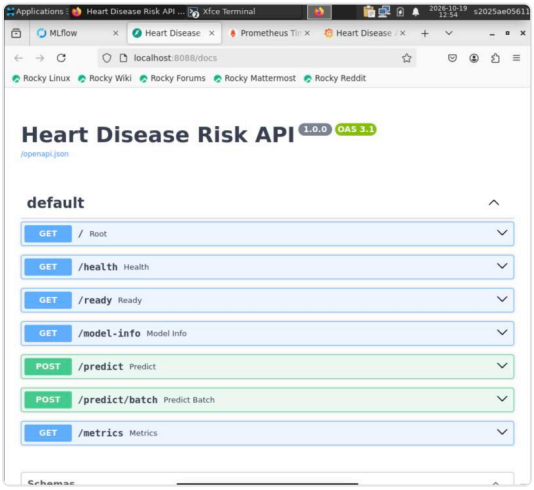


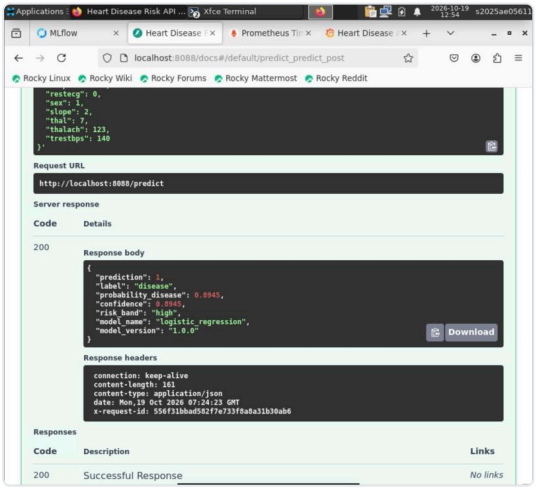




4.2 API through the Ingress

The FastAPI Swagger UI was served through the Kubernetes Ingress. A live `POST /predict` call executed from Swagger returned HTTP 200 with the prediction, confidence, risk band and an `x-request-id` header.

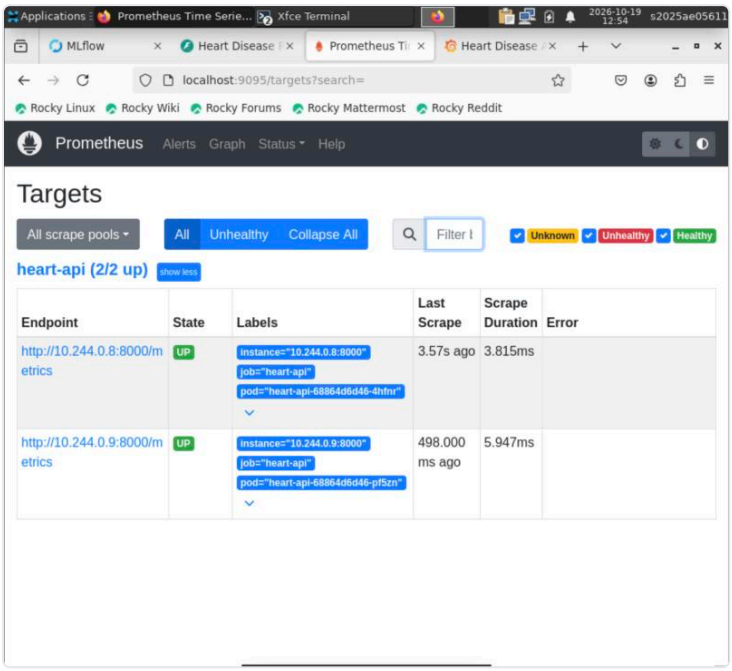




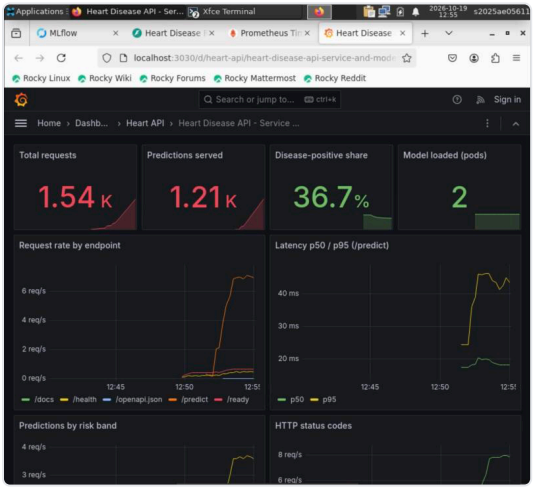
4.3 Monitoring: Prometheus and Grafana

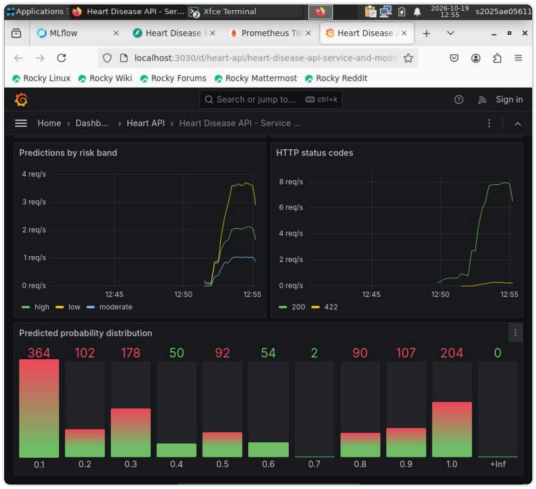
Prometheus discovered both API pods through their annotations and showed them as **UP (2/2)**. The provisioned Grafana dashboard was recorded while background traffic ran. It showed:

- about 1.5 K requests and 1.2 K predictions;
- 2 pods with the model loaded;
- request rate per endpoint and p50/p95 latency;
- predictions per risk band and HTTP status codes (200 and 422);
- the distribution of predicted probabilities.



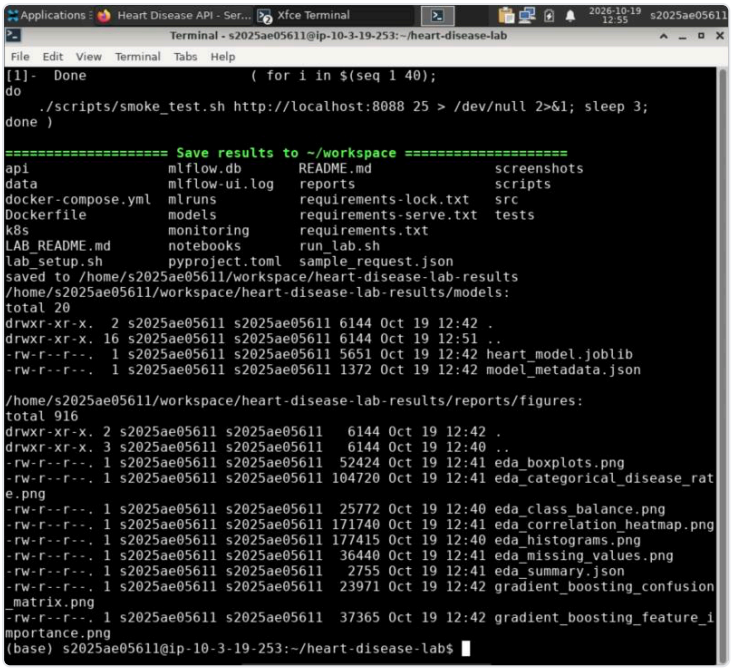
Prometheus targets





4.4 Results saved to persistent storage

`./run_lab.sh save` copied the trained model, model metadata, MLflow store and figures to `~/workspace/heart-disease-lab-results`, so they survive VM termination.



Results saved to ~/workspace

5. Reproducibility check: lab vs development machine

Metric (selected model: Logistic Regression)	Development run (macOS)	Prayogshala lab (Rocky Linux)
Best params	C=0.5, L2, balanced	C=0.5, L2, balanced
CV ROC-AUC (mean ± std)	0.9075 ± 0.0199	0.9075 ± 0.0199
CV accuracy	0.8469	0.8469
Test ROC-AUC	0.9589	0.9589
Test accuracy	0.8689	0.8689
Test recall / precision	0.9286 / 0.8125	0.9286 / 0.8125
Random Forest CV / test ROC-AUC	0.8961 / 0.9513	0.8961 / 0.9513
Gradient Boosting CV / test ROC-AUC	0.8839 / 0.9567	0.8839 / 0.9567
Unit tests	22 passed	22 passed
Sample `/predict` probability	0.8945	0.8945

All metrics are **identical to four decimal places** across a different OS, CPU architecture (ARM64 vs x86_64) and Python version (3.11 vs 3.12). This holds because the random seeds are fixed, the dependency versions are pinned, and all preprocessing lives inside the single serialised pipeline.

6. Issues encountered and how they were resolved

Issue	Cause	Resolution
Setup script assumed `apt`	The lab image is Rocky Linux (RHEL family)	`lab_setup.sh` detects dnf/apt and installs only what is missing
Setup ended early after installing tools	piping `kubectl version` into `head -1` closed the pipe early, and `set -o pipefail` treated that as an error	made the version printout tolerate the early close (`true` fallback)
`pip install` very slow	`~/workspace` is an AWS EFS (NFS) mount; a venv contains thousands of	ran from the VM's local disk; added a `save` step that copies results to

Issue	Cause	Resolution
	small files	`~/workspace`
Screen locked after a few idle minutes; the VM account password is not the BITS SSO password	XFCE screensaver with lock enabled	reconnected from the portal, then disabled the screensaver and lock for the session (`xfconf-query`, `pkill xfce4-screensaver`)
VM opened in a pop-up window	the portal's Connect opens a pop-up	opened the same VM address in a normal browser tab

7. Conclusion

The full MLOps pipeline ran end to end inside the BITS Prayogshala virtual lab, starting from a clean clone:

- environment setup;
- data acquisition and EDA;
- tuned training with MLflow tracking and registry;
- linting and 22 unit tests;
- Docker build and test;
- Kubernetes deployment with Ingress;
- endpoint verification;
- Prometheus and Grafana monitoring.

Every result matched the development run exactly. This shows the project is reproducible, portable across operating systems, and meets the assignment's production-readiness requirements: scripts run from a clean setup, the model serves correctly in an isolated container, and the pipeline fails fast with clear logs.